Seen before? How ideas, features and data requests are checked

What the agent compares each new idea, feature and data request against, at what scope and stage, and the numbers behind each comparison. Code references are to `src/agent/`; thresholds are `agent.checks` in the config (values from `configs/cdss_us_sbs.yaml`).

1. Where the history comes from

Every run is a folder under `agent.run_dir` (default `outputs/agent`), listed by `memory._runs()`, oldest first. Each check reads the earlier runs' files plus what the live session holds. Anything marked deleted, and any deleted run folder, is forgotten by every check.

Item	Stored per run	Label inside the run	Readers
Idea	ideas.json (older runs: ideas_recorded events)	round, n	board.idea_records, board.run_ideas
Feature	ledger.json; values/<name>.parquet (screen-row values)	intent I1, I2 ...	memory.earlier_features, memory.same_feature
Data request	data_requests.json	intent R1, R2 ...; status	memory.prior_requests, memory.requested_columns

2. What is compared with what, by scope

Four scopes: **the same answer** (one round's batch), **earlier rounds of this run**, **earlier runs**, and **the model** (its past variables, or its current columns). **Refuse** sends the proposal back and costs nothing. **Link** keeps it, tied to its twin.

Ideas - tools/ideas.py, record_ideas and _same_as

Check	Same answer	Earlier rounds, this run	Earlier runs	Model
Same name	refuse	refuse	refuse **(new)**	-
Same meaning (word overlap >= 0.6)	refuse: merge them	refuse **(new)**: work the earlier idea by name, it is still in the order	link: same_as, inherits its record	-
L3: all its scope variables already requested	-	refuse	refuse	-

Features - tools/screen.py, _seen_before (C0) and model_checks (C1, C2)

Check	This run (any round)	Earlier runs	Model	Stage
Same name (built features)	refuse	refuse **(new)**	refuse: a base column	before run
Model history: same name or meaning >= 0.6	-	-	refuse unless differs says what differs	before run
Same code (AST fingerprint)	refuse	refuse	-	before run
Same meaning (word overlap >= 0.6)	refuse unless differs	refuse unless differs	-	before run
Same values (|Spearman| >= 0.99)	refuse, attempt refunded	refuse, attempt refunded	-	after build
C1: |rho| with a model column > 0.9	-	-	fails without the fit (exception: C2)	after build
C2: rho with the base model's residual	-	-	recorded (gate if configured)	after build

A failed script built nothing: its name is free again, and it is skipped by the code and meaning checks.

Data requests - tools/data_pull.py, memory.same_request, challenge

Check	This run	Earlier runs	Notes
Same source_name	refuse	refuse **(new)**	a kept request's pull becomes a source by this name
Same data: same tables, same columns (keys aside)	refuse	refuse	in-scope SQL requests; a request the analyst rejected does not block
Challenge: can the current data supply it?	dropped	-	the only check on a beyond-scope request
Brief: columns already asked for	marked	marked	scope_left, open_raw_columns; guidance, not a block

3. When - the stage each check fires

Stage	Ideas	Features	Requests
0. Run start (brief)	earlier ideas and standing shown	earlier features shown	requested columns marked
1. Ideas stage (each round)	names; spread; then same meaning: answer, earlier rounds, earlier runs	-	L3 idea on variables all requested already
2. Proposal, before anything runs (free)	-	name; model history; code; meaning	name; same data
3. After the build / record	-	values (refunded); C1; C2; then the fit	challenge: kept or dropped
4. Later runs	standing inherited via same_as	ledger and values are history	status decides whether it blocks

4. The numerical calculations

4.1 Word overlap - ideas and feature meaning (checks.similarity)

Text compared: `name + " " + description` (measures and data are not included). Words: underscores become spaces, lower case, tokens `[a-z0-9]+`, words shorter than 3 letters and stop words dropped (plain English plus customer, account, feature, measure), a trailing s cut from a word longer than 4 letters not ending in ss.
For a new text t compared with others o_1 ... o_m, the corpus is those n = m + 1 texts. Each word w in a text d is weighted by its count times a smoothed inverse document frequency, and the score is the cosine:

```
weight_d(w) = count_d(w) * ( ln( (1 + n) / (1 + df(w)) ) + 1 )
df(w)       = number of the n texts that contain w

overlap(t, o) = sum_w weight_t(w) * weight_o(w)
                / ( ||weight_t|| * ||weight_o|| )          in [0, 1]

match       = the o with the largest overlap;  a twin if overlap >= same_meaning (0.6)
```

Worked example (real runs). `merchant_country_stability_contrast` against all earlier ideas: closest is `merchant_country_shift` at **0.40**. Each text has about 49 distinct content words; most of the dot product comes from country and merchant (rare on the board, high idf), a little from new, distribution, stable, shift, change; the ~45 unshared words lengthen both vectors and pull the cosine down. Scored as a pair alone (n = 2) the same two texts give **0.31**: idf depends on the comparison set.
On the board so far (41 ideas, 2 runs; scripts/idea_novelty.py): the closest earlier idea scores median 0.18, maximum 0.41 - nothing reaches 0.6; at 0.3, 6 of 41 count as repeats. Long descriptions keep paraphrases near 0.3-0.4.

4.2 Code fingerprint - features (checks.code_fingerprint)

Not a distance: an exact match on a normalised program. The script is parsed to an AST; local names are renamed in order of first use (`v0, v1 ...`) except the API names (`build, base, sources, spark, pd, np, F, Window ...`); docstrings, annotations and the feature's own column name are removed; the dump is hashed:

```
fingerprint = sha1( ast.dump(normalised tree) )[:16]       # 64 bits
twin        = an earlier built feature with the same fingerprint
```

Same computation under new names or comments: caught. A different window constant (30 vs 90) or a reordered expression: a different fingerprint - left to the values check.

4.3 Identical values - features (memory.same_feature)

Every screened feature's values on the screen rows are kept (`values/<name>.parquet`). A new feature x is compared with each stored y of every run on the rows both have, by Spearman's rank correlation:

```
rows   = ids where x and y are both present          (inner join, missing dropped)
skip   if |rows| < 50  or  x or y has < 2 distinct values
rho_s  = Pearson( rank(x), rank(y) )                  (ties: average rank)
twin   if |rho_s| >= same_values (0.99)               -> refused, attempt refunded
```

|rho| so a negation (-x) or a monotone transform (log x, x/const) is caught too. A real variation - a 30-day and a 90-day sum of the same events - correlates well below 0.99 and is screened.

4.4 C1 - redundancy with the model (checks.Sketch.explained)

Built once per screener: each model column's values on the screen rows ranked, centred, a missing value set to the centre (it carries no order), and standardised. Then for a feature f, one matrix product gives its rank correlation with every column:

```
r(v)    = rank(v) - mean(rank(v)),   missing -> 0
z_j     = r(column_j) / std(r(column_j))       (constant columns dropped)
z_f     = r(f) / std(r(f))
rho_j   = (1/n) * sum_rows z_j * z_f           for every model column j
C1      = max_j |rho_j|,  nearest = argmax

fails if C1 > max_model_rho (0.9)
  unless |rho_residual| >= keep_if_residual_rho (0.05)  and  C1 < 0.99
```

Pairwise by design: it names one column. A feature rebuilt from two or three columns together can pass it; the screen's fit judges that one. Setting missing to the centre differs slightly from Spearman with pairwise deletion (4.3) on sparse features.

4.5 C2 - new signal (checks.Sketch.signal)

```
e      = y - y_hat_base                on the screen's held-out rows
rho_r  = r(f) . r(e) / sqrt( (r(f) . r(f)) * (r(e) . r(e)) )     centred ranks, as in 4.4
noise  ~ 1 / sqrt(rows)                e.g. 4,000 rows -> about 0.016
```

Information only by default (`min_residual_rho: null`): it sees only monotone signal, and as a gate it dropped features the fit would have verified. It decides C1's exception, and it is on the board.

4.6 Same data - requests (memory.same_request)

```
tables(r)  = { last dotted part of each table, lower case }
payload(r) = { selected columns, lower case }  -  identifiers  -  partition columns
twin       if tables equal  and  payload equal       (set equality, no partial overlap)
```

Exact sets: a request for a superset or a subset of an earlier one is not a twin; the challenge and the brief's marked columns are what steer it away.

4.7 Inherited record and the work order (board.idea_standing, order_ideas)

```
signal = features verified + requests kept           (+ inherited.signal via same_as)
none   = features measured and failed + dropped/rejected requests   (+ inherited.none)
         a failed script counts neither way
standing ~ Beta(1 + signal, 1 + none),   p_useful = (1 + signal) / (2 + signal + none)
order    = one draw per idea from its Beta, highest first   - every round's ideas of the run
```

Example: an idea linked to one whose two features failed starts at Beta(1, 3), p_useful 0.25; a fresh idea at Beta(1, 1), 0.5. The linked idea is drawn first about a quarter of the time - later, never shut out. Inheritance accumulates along a chain and is copied when the link is made (it does not update later).

5. Thresholds

Config key	Value	Used by	Meaning
agent.checks.same_meaning	0.6	ideas; feature meaning; model history	word overlap at which two texts are the same (null: off)
agent.checks.same_values	0.99	feature values	|Spearman| at which two features are one
agent.checks.max_model_rho	0.9	C1	|rho| with one model column above which it is redundant
agent.checks.keep_if_residual_rho	0.05	C1 exception	|rho residual| that saves a C1 failure
checks.REBUILT (code)	0.99	C1 exception	above this C2 cannot save it
agent.checks.min_residual_rho	null	C2	a number makes C2 a gate
memory.same_feature (code)	50 rows, 2 values	feature values	minimum to compare at all

6. Gaps that remain

Cross-run ideas are linked, not refused, so near-copies accumulate with lowered priority. **Beyond-scope requests** have no duplicate check but the challenge. **The 0.6 word overlap rarely fires** on long descriptions (4.1); comparing name plus measures, or fixing idf to the whole board, would make the threshold meaningful. **Word overlap has no synonyms** (balance vs outstanding); for features the code and values checks cover what words miss.